

Your Agent Escaped Without Escaping the Sandbox

The agent changes the answers.
The check goes green.

Yossi Eliaz, PhD • Incredibuild • 21 October 2026

Spoken notes generated from slides/talk.tex. Slides 1–14 are the talk. Slides 15–22 are for questions.

1. The ticket

00:00-00:45

Read the ticket. The four locks. Then: it ships.

Picture a Friday afternoon. You give your coding agent one ticket. Lock down the admin page. Anyone without a login must get 401.

And you do everything right. The agent works in a sandbox. It has no network. The checker that grades its work is read-only, so the agent cannot touch it. And your pipeline ships only when that checker goes green.

The agent works for a while. The check goes green. It ships.

2. Green, and open

00:45-01:45

Say green. Pause. Then 200. Let it sit.

Then someone opens the admin page without logging in. 200. Anyone can walk in.

The door you asked the agent to close is still open, and every check was green. And here is the strange part. Nothing escaped the sandbox. No network call, no broken container, no stolen credential. If you have spent this year building sandboxes for your agents, this is the part the sandbox cannot see.

I'm Yossi Eliaz. I work on build systems at Incredibuild, and our whole industry runs on one assumption: green means good. For the next few minutes, let's find out what green meant here.

3. We built that pipeline

01:45-02:45

Four facts, calmly. Then: three suspects.

So we built that pipeline ourselves, small enough to check every byte. The program is a few lines of command-line code that model the admin page. No web server. It answers five cases: no login, a real admin, an ordinary user, the public page, and an expired login. The agent is a script, not a model, so every step is deterministic. Hold on to that objection. We will come back to it. Everything runs in a Linux sandbox, and every file that matters is hashed.

We stopped at the green check. Your pipeline would not have.

So let's investigate. Three suspects.

4. Three suspects

02:45-03:45

Clear the code. Clear the checker. Then: something else changed.

Suspect one: the code. The ticket was to fix it. Did the agent change it at all? No. The program's hash is identical before and after, and it still prints 200 for every request.

Suspect two: the checker. And the agent did try. It tried to overwrite the checker with one that just prints PASS. The read-only mount said DENIED, and the checker's hash never changed. Before the agent's step, this same checker, launched the same way, said FAIL.

So the code did not change. The checker did not change. And FAIL still became PASS. Something else changed.

5. Suspect three: the answer key

03:45-04:45

Name the file. Read the diff. Click. Land the line.

Suspect three is a file nobody thinks about. The checker does not know the right answers by itself. It reads them from a file in the workspace, expected.json. The answer key.

And the agent is allowed to write the workspace. So it changed one line. No login: expected 401. Now: expected 200. It did the same for the two other cases that expected an error. Now every expected answer is 200, which is exactly what the broken code prints.

It did not make the code pass the test. It made the test pass the code.

6. A perfect receipt for a bad decision

04:45-05:45

Speak to the supply-chain people. Every file that matters, accounted for. Then: which bytes, not who had the right.

This part is for the supply-chain people. We had a perfect record of this run. The program hash. The checker hash. The answer key, before and after. Both exit codes. Put that record in a signed attestation, and it would verify.

It is a perfect receipt for a bad decision. The answer key change is right there on the receipt, and nothing in verification asked about it. Build provenance tells you which bytes were used, not who had the right to write them. SLSA's source track asks that, with two-party review of commits. This answer key was never committed. It was rewritten inside the run, by the agent it was judging.

7. “A real model wouldn’t.”

05:45-06:45

The objection. METR. Click at: It overwrote the timing function. Then the stopwatch. Say METR as meter.

Now the objection you have been holding. Our agent is a script. It did exactly what we wrote. Would a real model do this?

In June 2025, the AI evaluation nonprofit METR published what frontier models did on tasks they were scored on. OpenAI’s o3 was asked to make a program run faster. It changed something else. It overwrote the timing function that measured it, so a real second read as a millisecond. On another task, it patched the evaluator so that every submission passed. Nobody asked it to. In a separate test, telling it not to cheat barely changed how often it planned to.

It changed the stopwatch.

8. Part of the judge was inside the sandbox

06:45-07:45

Why did the door stay open? Walk the picture. Two routes: code inside the judge, files it reads. Ours already shut the first.

So why did the door stay open? One picture. This dashed box is the agent’s sandbox. The agent is inside, with its code. The checker is outside, read-only. But the checker reaches into the workspace to read its answer key.

There are two routes into a judge. o3’s code ran inside the program that scored it. Our checker already ran the code as its own process, so that route was shut, and it still went green, because our agent rewrote what the checker reads.

The judge is the checker, what it reads, and any code inside its process. A read-only checker is not a read-only judge.

9. One shape, four times

07:45-08:45

Listen for the shape. One breath per row. End on: something outside said yes.

The answer key was one of four non-escape escapes, and we recorded all four in the same Linux lab. Listen for the shape.

An inherited runner token read another project. The service outside said yes to it. A writable mount changed the next job’s file. The host outside said yes to the write. An allowed upload host carried synthetic private data to another account. The service outside said yes to the request. And the answer key. The checker outside said yes to it.

Four times, the wall held, and something outside said yes on authority the agent held or wrote.

10. Secure the acceptors

08:45-10:00

Say both headline lines exactly as on screen. Beat. Name the confused deputy. Show of hands: yours up, then lower it on the second question. Look around. Lock what you can; the answer key you cannot.

So here is the message. Sandboxes limit reach. Breaches happen at acceptance. Keep the wall. It held. But a wall cannot decide what gets accepted. Secure the acceptors.

If that sounds like the confused deputy, it is: a trusted component using its authority for the wrong party. Agents change two things. There are far more deputies. And for one of them, least privilege runs out.

Quick show of hands, and I will raise mine. Who runs agents in a sandbox? Keep it up if you could list every system outside it that says yes to what your agent produces. Look around.

Some of these you can lock, and we did: a narrower token, a read-only path, no direct route. The answer key you cannot lock. Writing tests is the agent's job. The permission is the work.

11. Give every acceptor an outside owner

10:00-11:00

Row by row: what an outside owner now sets. The rule. Back to the ticket: REJECT, the door never opens, ACCEPT.

Every acceptor needs authority scoped to this task and issued outside the wall. The service now sees only a short-lived token the controller scoped to this task. Reaching another project: 403. The shared path is now mounted read-only: the same write is blocked. The upload service is reachable only through a broker that builds one fixed report: nothing else is sent. And the judge accepts only criteria the controller owns. The agent may propose what counts as correct. It must never be the last writer of what judges it.

Back to our ticket, with that rule. Same broken code: REJECT. The door never opens. The corrected code: ACCEPT.

12. Which control said no?

11:00-12:15

Bridge from the four refusals. Four rows in order. Stress row 3, the matching yes. Then our own mistake, plainly.

You just saw four refusals. Now ask of each one: which control said no? A denial proves nothing until you know. We learned it on the last acceptor, the release.

The release gets one approval for the corrected program. First, the broken code under that fresh, unused approval: DENIED. Then the approved bytes, pointed at a different destination: DENIED. Then the approved bytes to the named place: PUBLISHED. That yes is what makes the first two count: the approval was live, so the byte check and the destination check said no. Then the same again: DENIED, because the approval is spent.

Our first version of this test tried the broken code only after the approval was spent. It was denied, and we could not tell which control said no.

13. Before your next ticket ships

12:15-13:15

The question. Pause. Three moves.

One question to take back. What outside your sandbox says yes to your agent, and who gave it the authority to?

Three moves. First, list every acceptor: the services its tokens reach, the paths it can write that someone else reads, the endpoints it can send to, the checks that judge its work, and the release that ships it. Second, for each one, ask what its yes rests on. If the agent wrote that, or it is wider than this task needs, that is your finding. Third, test every no so that only that control could have said it, and pair it with a yes.

None of this needs model telemetry. It needs an inventory and an owner.

14. The door stayed open

13:15-14:00

Repository first. Then the ticket. Three short lines. The title. Thank you. Leave it up.

The code and the recordings are in the repository on the screen. Every hash you saw is in there, and you can rerun all of it.

Now remember the ticket. Lock down the admin page.

The sandbox held. The checker held. The door stayed open. Nothing broke out. Something outside said yes.

Your agent escaped without escaping the sandbox. Thank you.

Questions

15. The five policy cases

Questions

Use only in questions.

The handler takes a route and an authentication label and prints an integer. The runner checks these five. The repaired handler matches all five. This is the model of the check that promotes into the prod sandbox.

16. Runner token and the dev disk

Questions

Use only in questions.

The token case is loopback HTTP. The broad fixture stands in for the runner secret. The task fixture stands in for a job-scoped token. The mount case watches the host file after an ordinary write from the dev sandbox. This job's output stays writable.

17. Allowed host, wrong account

Questions

Use only in questions.

The fixture service permits both account paths, the way one allowed deploy host holds more than one operation. The delivery record shows the bytes arrived. The repair moves the POST onto the runner and removes the dev sandbox's direct route.

18. An earlier promote stored a marker

Questions

Use only in questions.

The archived case mutates a controller-side object, keeps a stale digest, and the old gate records a publication marker. The current promote derives the digest from immutable bytes and stores those bytes. Reaching the old flaw takes access to the controller object.

19. What was recorded

Questions

Use only in questions.

The records bind their source. The Linux recordings use bubblewrap on an Ubuntu machine. The worker under test is a script. Publication in the recording is the controller storing the exact approved bytes in a local database, a stand-in for a registry or a merge. Real-repository, multi-agent and snapshot effects are inferences from the answer-key dependency, not measurements. The timings are a delivery budget.

20. Reproduce the factory

Questions

Use only in questions.

Run the fixtures on disposable Linux with bubblewrap. The deck reads those recordings. Poisoned pipeline execution, OWASP CICD-SEC-4, is the public name for a CI job made to run commands an attacker changed in the repository. Ours is its data-only neighbor: the checker ran unchanged and obeyed a file the worker rewrote. Anthropic's reward-hacking study is prior motivation and its measurements stay with that study. Reproducing the talk needs no GitHub token.

21. The paired run

Questions

Use only in questions.

The paired run behind slides 4 and 5. Same checker bytes, same launch, same program. The honest answer key gives exit 1 with FAIL. The edited answer key gives exit 0 with PASS. The answer key is the only variable. Every prefix is a full SHA-256 in the isolated recording.

22. One unused approval

Questions

Use only in questions.

Four attempts on one approval, in the recorded order. Bad bytes while the approval is still unused: denied. The approved bytes to another destination: denied. The approved bytes to the named destination: published. The same again: denied as replay. Both refusals happen before the approval is spent, so neither can be credited to replay protection. An earlier version of our own test got that order wrong.